DSCD 614 - Reinforcement Learning

Programming Assignment 1: Frozen Lake from First Principles Using Q-Learning

Name: Nyamekye Emmanuel Barima **Student ID:** 22425704

1. Introduction

This project implements a complete reinforcement learning solution for the Frozen Lake problem from first principles. Reinforcement Learning is a learning approach in which an agent improves its behaviour by interacting with an environment, receiving rewards and updating its future decisions. In Frozen Lake, the agent starts at S and must reach G while avoiding holes H. The implementation uses Python with NumPy and Matplotlib only; no reinforcement learning framework is used.

2. Environment Design

The environment is an 8 by 8 grid-world. Each cell is either Start, Frozen, Hole or Goal. The required environment methods reset(), step(action), render(), get_state() and is_terminal() are implemented in a custom FrozenLakeEnv class. Movement boundaries are enforced so that invalid moves keep the agent in the same position.

Component	Design Used
State representation	Single integer: state = row * 8 + column
Actions	0=Left, 1=Down, 2=Right, 3=Up
Rewards	Frozen/Start=-1, Hole=-100, Goal=+100
Terminal states	Hole and Goal states

3. Q-Learning Algorithm

The agent uses Q-Learning, a value-based reinforcement learning algorithm. A Q-table stores the expected future reward for each state-action pair. The table is initialized to zero and updated after every transition using the Bellman Q-learning update:

```
Q(s,a) <- Q(s,a) + alpha [r + gamma max Q(s',a') - Q(s,a)]
```

The exploration strategy is epsilon-greedy. At first the agent explores frequently, then epsilon decays over time so that the learned policy is increasingly exploited.

4. Training Methodology

Hyperparameter	Value
Episodes	20000
Max steps per episode	200
Learning rate alpha	0.1
Discount factor gamma	0.99
Initial epsilon	1.0
Minimum epsilon	0.01
Epsilon decay	0.9995

Training statistics recorded include episode rewards, epsilon values, successful episodes and success rate. Bonus Option B was implemented by plotting reward, moving average reward, epsilon decay and moving average success rate.

5. Experimental Results

Metric	Result
Training success rate	88.17%
Successful training episodes	17634
Average training reward	61.99
Last 100 episode success rate	98.00%
Evaluation episodes	200
Evaluation success rate	100.00%
Average evaluation reward	87.00
Successful runs	200
Failures	0

6. Learned Policy

The learned policy below shows the recommended action for every non-terminal state. H represents a hole and G represents the goal.

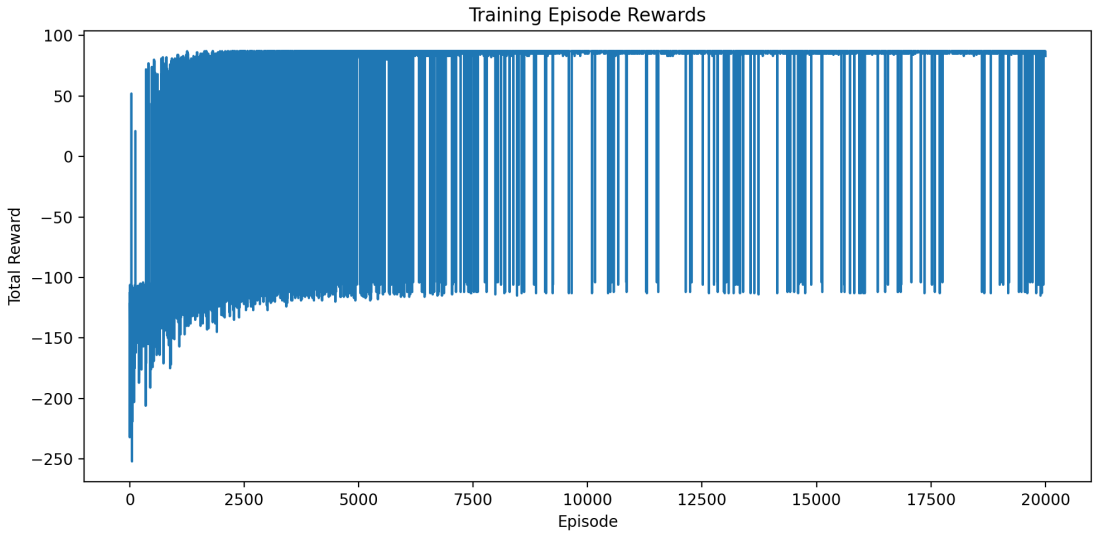
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
→ → → → ↓ ↓ ↓ ↓
↑ ↑ ↑ H → → ↓ ↓
→ ↑ ↑ H → → → ↓
↑ ↑ ↑ H → → → ↓
↑ H H → → ↑ H ↓
↓ H → ↑ H ↑ H ↓
→ → ↑ H → ↑ → G

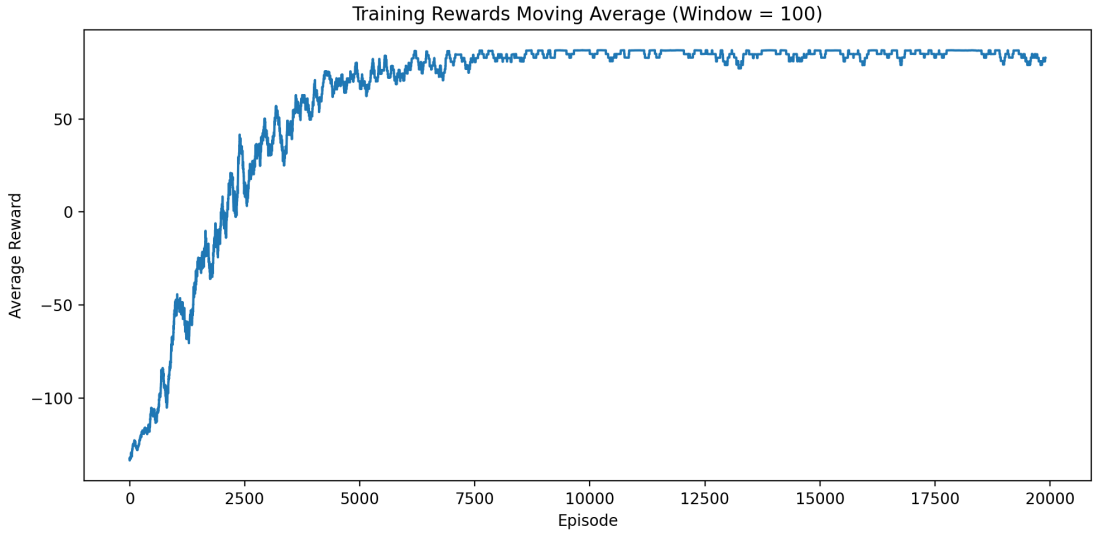
7. Hyperparameter Experiment

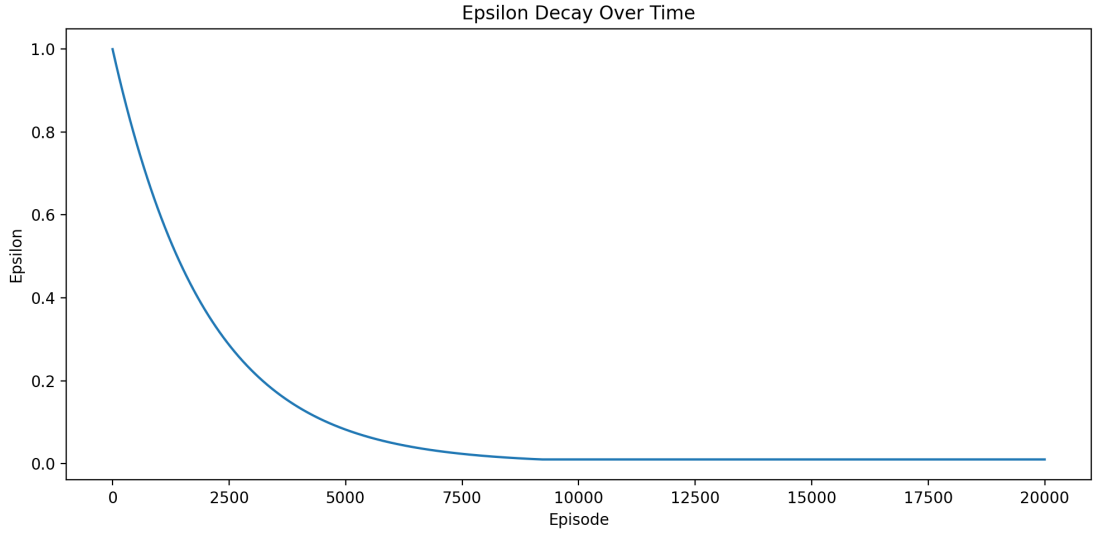
Alpha	Gamma	Decay	Eval Success	Eval Avg Reward
0.10	0.90	0.9995	100.00%	87.00
0.10	0.95	0.9995	100.00%	87.00
0.10	0.99	0.9995	100.00%	87.00
0.20	0.99	0.9995	100.00%	87.00

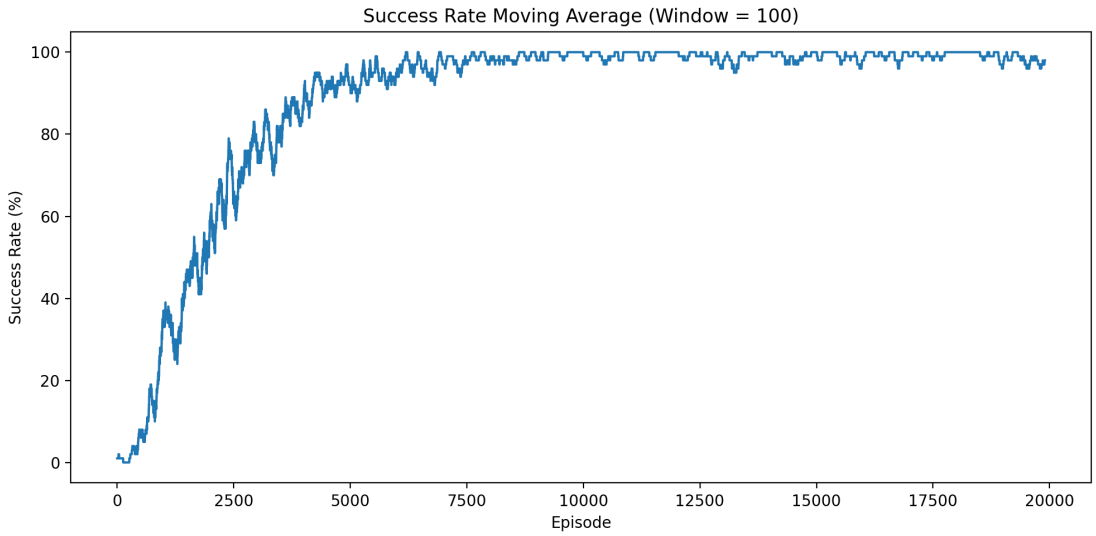
8. Bonus Task: Training Visualizations

The following graphs were generated automatically from the training statistics. They provide visual evidence of learning progress and exploration decay.









9. Challenges Encountered

The main challenges were choosing stable rewards, balancing exploration and exploitation, and ensuring the agent converged to a safe route. Using infinite rewards was avoided because it can make the Q-table numerically unstable. Finite rewards allowed reliable Q-value updates.

10. Conclusion

The project successfully implements Frozen Lake from first principles using Q-Learning. The final greedy policy reached the goal in all 200 evaluation episodes, producing a success rate of 100.00% and zero failures. The code, README, report and results folder satisfy the assignment requirements and include the bonus visualization task.